URL Shortener System - Requirements

Functional Requirements

• Users can submit a long URL and receive a unique short URL.

• Redirect users from a short URL to the original long URL.

• Generate unique short codes using Base62 encoding or auto-increment IDs.

• Store short URL to long URL mappings in the database.

• Use Redis cache to reduce database lookups.

• Support high read traffic for redirection.

• Record click analytics (timestamp, IP, browser, device, location).

• Allow optional custom aliases.

• Support URL expiration and deletion.

• Provide REST APIs for URL creation and retrieval.

Non-Functional Requirements

• High availability (99.9%+ uptime).

• Low latency (<100 ms for redirects).

• Horizontal scalability using load balancers.

• Reliable data storage with backups.

• Fault tolerance using replication.

• Security through HTTPS, input validation and rate limiting.

• High throughput (millions of requests/day).

• Caching using Redis for frequently accessed URLs.

• Asynchronous analytics processing using a message queue.

• Easy monitoring, logging and maintenance.

APIs

POST /url - Create Short URL

GET /{shortUrl} - Redirect to Original URL